Project - Low Level Design

on

Enerlytics AI

Course Name: GEN AI

Institution Name: Medicaps University – Datagami Skill Based Course

Sr no	Student Name	Enrolment Number
1.	Riya Patidar	EN22CS301813
2.	Ritik Patidar	EN22CS301809
3.	Roshni Patidar	EN22CS301825
4.	Srashti Bhadoriya	EN22CS301982
5.	Pranjal Sharma	EN22CS301731

Group Name: 10D7

Project Number: GAI-33

Industry Mentor Name: Aashruti Shah

University Mentor Name: Dr. Hemlata Patel

Academic Year: 2025-2026

Table of Contents

Chapter 1	Introduction	3-4
	1.1. Scope of the Document	3
	1.2. Intended Audience	3
	1.3. System Overview	4
Chapter 2	System Design	5-9
	2.1. Application Design	5
	2.2. Process Flow	6
	2.3. Information Flow	7
	2.4. Components Design	7
	2.5. Key Design Considerations	8
	2.6. API Catalogue	9
Chapter 3	References	10

1. Introduction

1.1 Scope of the Document

This document provides a comprehensive Low-Level Design (LLD) for the **Enerlytics AI** platform, an advanced Retrieval-Augmented Generation (RAG) system specialized for the energy sector. The primary focus is to outline the proposed internal structure, including architectural components, data orchestration, and the integration of Large Language Models (LLMs) with high-density vector databases.

The scope encompasses detailed technical blueprints for the automated ingestion pipeline, the domain-grounding logic for energy technical assets, and the semantic retrieval mechanisms used for generating high-fidelity AI responses. It serves as a foundational guide for understanding the interaction between the FastAPI backend, the React-based neural interface, and the persistence layers (Redis and Qdrant).

1.2 Intended Audience

This document is designed for system architects, lead developers, and technical stakeholders involved in the evaluation and future development of the energy intelligence platform. It provides a granular look at the Generative AI implementation strategies and the infrastructure requirements for scalable document analysis.

1.3 System Overview

Enerlytics AI is an intelligent RAG-driven platform created to assist grid operators and energy specialists in navigating complex technical documentation. Unlike generic AI bots, this system is designed to provide domain-specific insights by leveraging local embedding models and high-performance LLM routing.

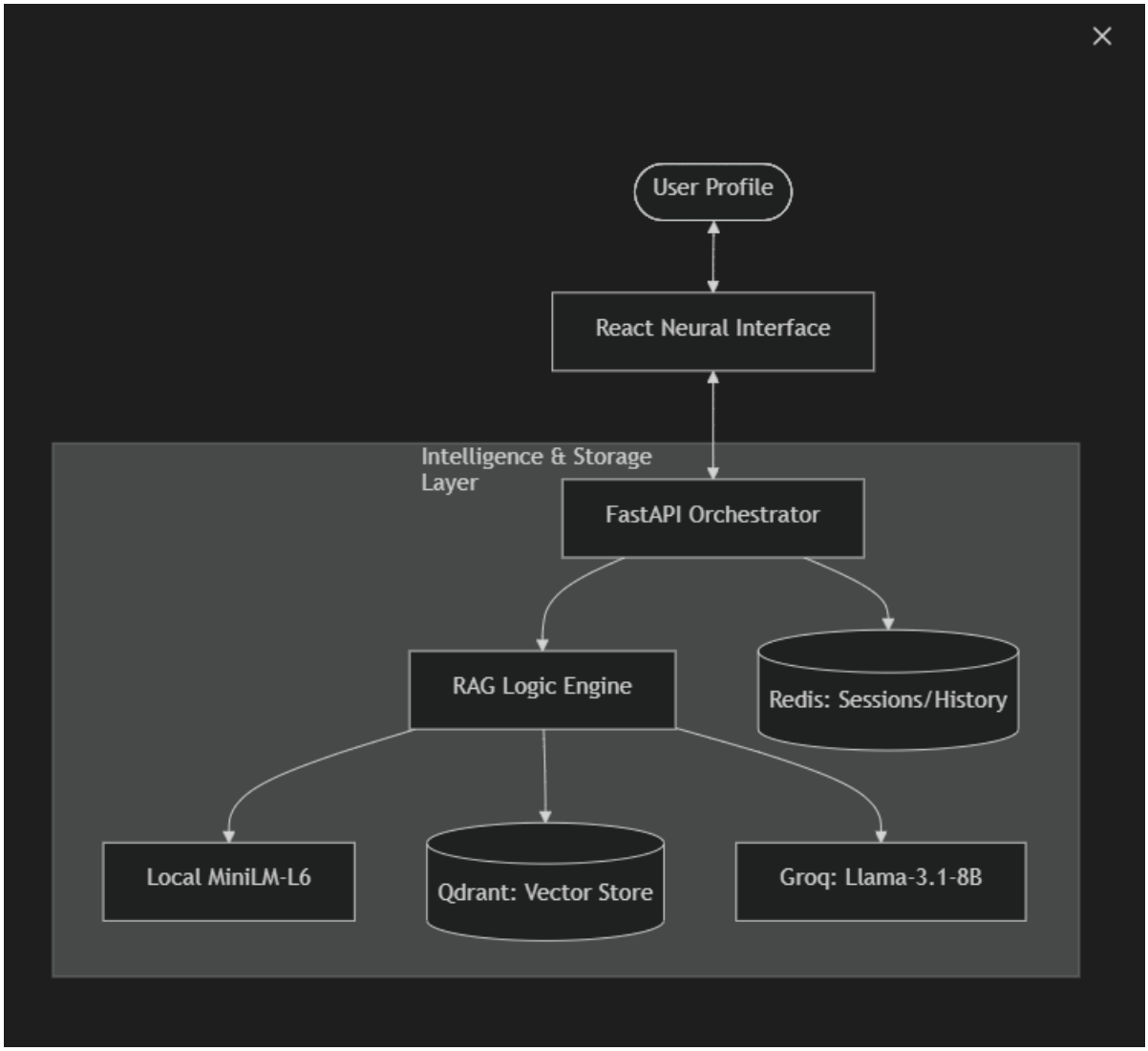
The core of the system will be built around a multi-stage validation engine that filters for energy-related keywords before any vector processing occurs. Through the use of vector databases, the bot will retrieve relevant context from technical blueprints and load reports to ensure that AI-generated responses are accurate and grounded. The proposed backend

architecture utilizes Python and FastAPI to ensure low-latency communication during streamed analytical queries.

2. System Design

2.1 Application Design

The application is conceived with a modular, layered architecture to facilitate scalability and maintainability. The proposed architecture follows a structured pipeline:

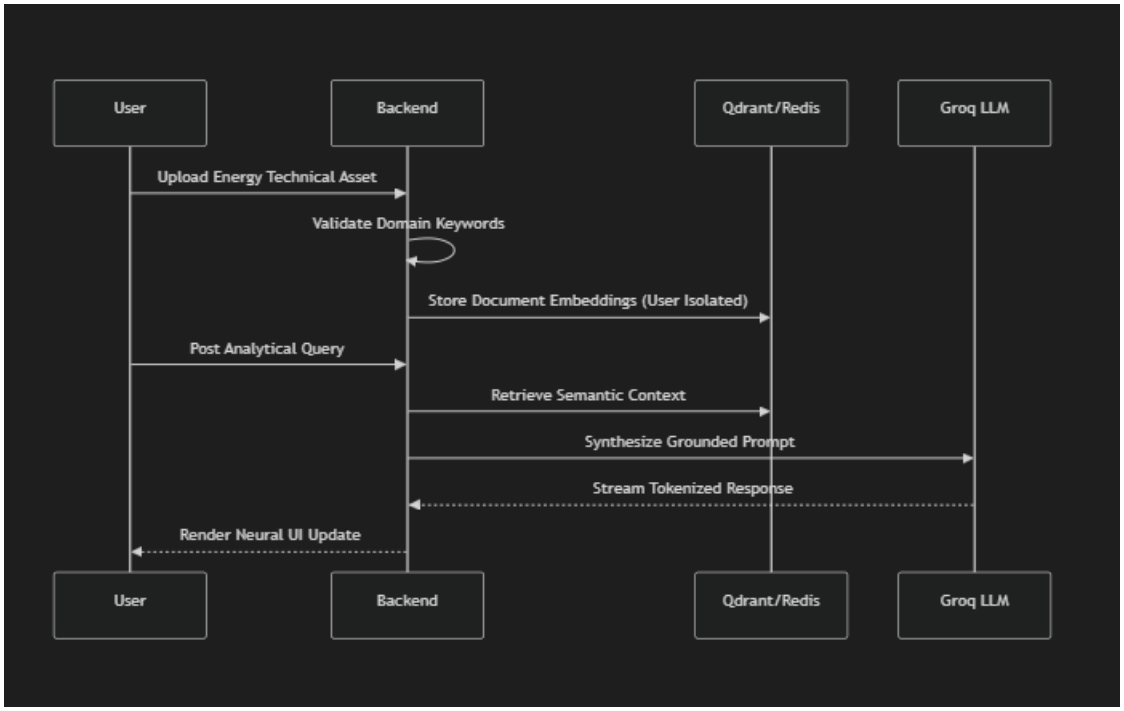


The system will be partitioned into three primary layers:

- **Presentation Layer (React):** Handles the neural-themed user interface, including real-time message streaming and analytical trace visualization.
- **Backend Orchestration (FastAPI):** Serves as the central nerve center responsible for session management, user authentication, and coordinating communication between the AI and storage layers.
- **Intelligence Layer (RAG Pipeline):** Manages the lifecycle of document embeddings, vector retrieval from Qdrant, and response synthesis via the Groq LLM API.

2.2 Process Flow

The system's process flow ensures a secure "Upload-to-Insight" loop as visualized below:



The system's process flow is designed to ensure a seamless "Upload-to-Insight" loop. Initially, the user initiates an ingestion session by uploading energy-related technical assets. The system will perform an immediate "Domain Validation" check; if valid, the document is chunked and vectorized using a local MiniLM-L6 model.

When a query is received, the backend will generate a semantic search vector and retrieve the relevant context from the vector store. This context, along with current chat history from Redis, is formatted into a prompt for the LLM. The final result is streamed back to the user

via a persistent connection, allowing the interface to render the AI response word-by-word for improved perceived latency.

2.3 Information Flow

Data within the system flows in a structured, uni-directional loop to maintain integrity. User inputs trigger requests to the vectorized storage to find high-similarity "evidence." This evidence is then fused with the user's prompt and passed through a secure LLM pipeline using a "System-Persona" grounding message. The resulting information is processed by the backend to ensure domain grounding before reaching the user interface. Session persistence will be managed via Redis, ensuring that conversation history is contextually preserved during the 10-day session lifecycle.

2.4 Components Design

- **Document Ingestor**: Responsible for PDF/DOCX extraction and the initial keyword-based filtering logic.
- **Vector Manager**: Orchestrates the local embedding generation and communication with the Qdrant point-storage.
- **Neural Chat Engine**: Manages the prompt engineering and response streaming from the Groq API.
- **Chat Interaction Module**: A frontend component designed to handle real-time message state. It will implement an "Optimistic UI" approach, showing user messages instantly while buffering streamed AI tokens. It also includes the **Analytical Trace Toggle**, which allows users to audit the grounding documents in a separate modal view.
- **Auth Service**: Implements JWT-based secure session management and cookie-level persistence.

2.5 Key Design Considerations

- **Accuracy & Grounding**: The design prioritizes "Hallucination Protection" by strictly limiting the LLM to the provided energy context.
- **Performance**: The use of local embeddings and a reverse-proxy (Nginx) ensures fast response times in various network conditions.
- **Data Isolation**: The logic ensures that every retrieval operation is strictly filtered by unique user_id metadata.

2.6 API Catalogue

The system will expose a set of standardized RESTful APIs including:

- **/auth/signup:** Optimized onboarding and workspace initialization.
- **/api/upload:** Neural document analysis and ingestion.
- **/api/chat:** Multi-modal chat interface with streamed response support.
- **/api/workspace/init:** Unified session recovery and environment synchronization.

3. References

1. **FastAPI Documentation**: https://fastapi.tiangolo.com/
2. **Qdrant Vector DB Guidance**: https://qdrant.tech/documentation/
3. **Groq Llama-3 API**: https://console.groq.com/docs/
4. **Redis Persistence Layers**: https://redis.io/docs/
5. **Nginx Reverse Proxy Config**: https://nginx.org/en/docs/